1 Lesson 30-10-26

1.1 RTL design process

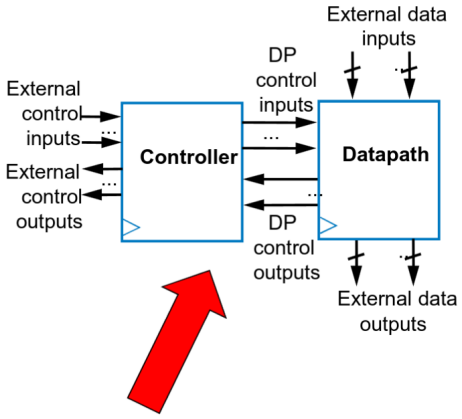
We know get to actual **RTL** (*Register Transfer Level*) design.

The idea is to:

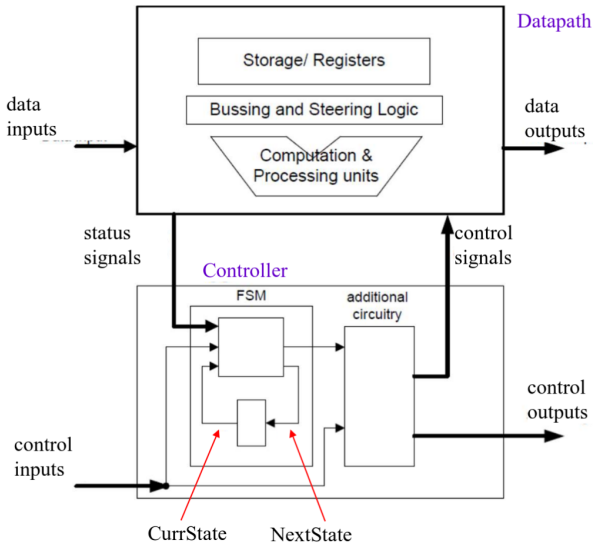
- Capture behavior (via a behavioral**HLSM**);
- Convert the behavior to a circuit: we need a target architecture, that is a *datapath* capable of the HLSM's data operations, and a *controller* able to control the datapath.

The model that works for us is the **FSM-D** model: *FSM with Datapath*.

- The **controller** is a classical FSM, easy to design with our known behavioral models, easy to synthesize;
- The **Datapath**, given in a structural or behavioral description, contains the functions controlled by the datapath.



Note that the datapath is not necessarily only combinatorial: it can contain sequential components (as we saw earlier with *Sequential Datapath Compoenents*).



Let's now look at the actual design procedure.

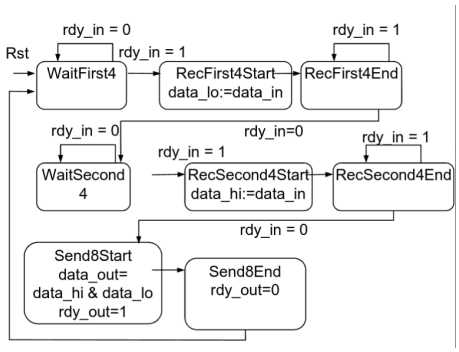
1. Describe the system's desired behavior as a high-level state machine. The state machine consists of states and transitions. The state machine is "high-level" because the transition conditions and the state actions are more than just Boolean operations on single-bit inputs and outputs;
2. Create a datapath to carry out the data operations of the high-level state machine;
3. Connect the datapath to a controller block. Connect external control inputs and outputs to the controller block;
4. Convert the high-level state machine to a finite-state machine (FSM) tor the controller, by replacing data operations with setting and reading of control signals to and from the datapath.

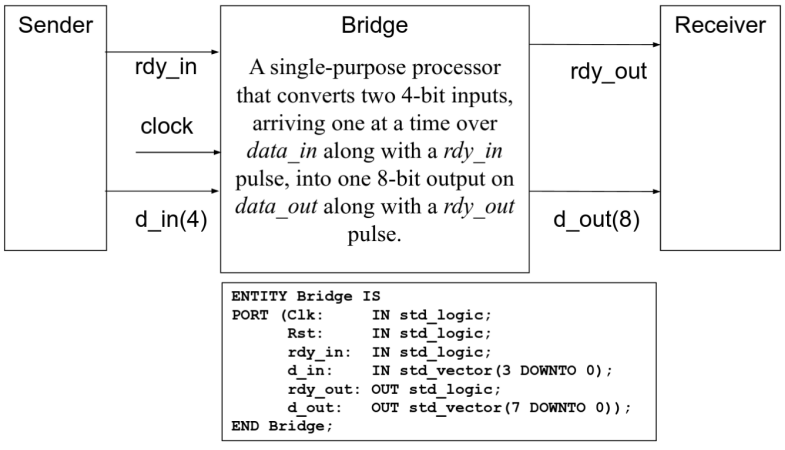
We now introduce two categories of FSM-D design.

- In **control dominated design** we have complex controller controlling a simple datapath.

  Let's take, as an example, a *bridge* component. This is a single-purpose processor that converts two 4-bit inputs, arriving one at a time over `data_in` along with a `rdy_in` pulse, into one 8-bit output on `data_out` along with a `rdy_out` pulse.

  Explained via the state diagram:






```
ENTITY Bridge IS
PORT (Clk:     IN std_logic;
      Rst:     IN std_logic;
      rdy_in:  IN std_logic;
      d_in:    IN std_vector(3 DOWNTO 0);
      rdy_out: OUT std_logic;
      d_out:   OUT std_vector(7 DOWNTO 0));
END Bridge;
```

VHDL: Embedded Systems RTL Design

- The alternative is **data dominated design**, where we have an extensive datapath and a simple controller. A good example is a filter, which converts digital input stream to new digital output stream, for instance to remove *noise*.



A simple filter gives as an output the weighted sum of the last N values. Small N means less filtering, and large N means more filtering, but less sharp output.

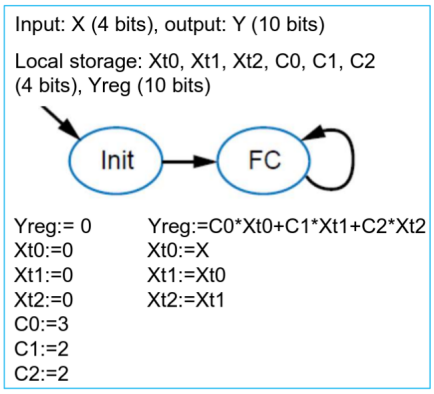
If the weights of the sum can be chosen, then this is a **FIR** (*Finite Impulse Response*) filter. Simply a configurable weighted sum of past input values:

$$Y(t) = W_0 \cdot X(t) + W_1 \cdot X(t-1) + C_w \cdot X(t-2)$$

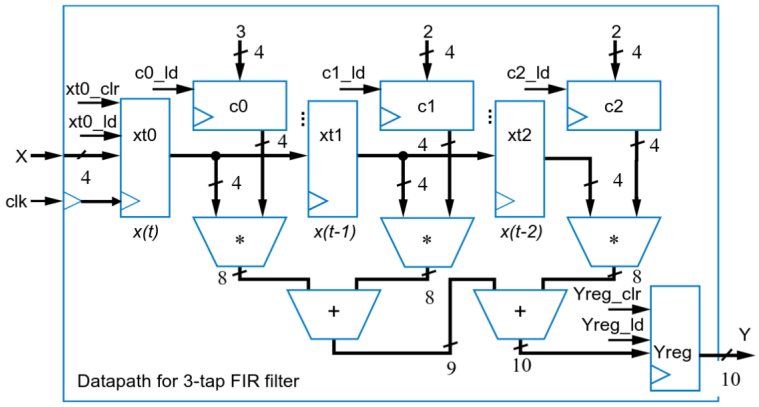
This is known as a *3-tap* filter, as it involves 3 multiplicatory terms. Tens of taps per filter are more common. Know that this is a very general filter: the designer sets the constants (W_0, W_1, W_2) to define a specific filter behavior.

From the hardware standpoint, this is simply a set of multiplication circuits, plus an adder for all terms.

The FSM is trivial, as mostly we want to keep state in our storage devices (for each sampled tap value).



Now, the datapath for this is far more complex, because the operations we seek to implement is arithmetically more complex:



1.1.1 Critical paths

Designers of digital circuits often want the fastest possible performance. This means high clock frequency.

Frequency is limited by the longest register-to-register delay in our datapath, known as the **critical path**. If clock is any faster, incorrect data may be stored into our registers.

1.2 Algorithmic-Level Behavior

Good news! An abstraction level higher than RTL.

HLSM was already pretty high-level, but even higher is sometimes needed for certain applications. In an **algorithmic-level** description, behavior is described as sequence of steps, similar to sequential program.

Video compression, for example, computes sum of absolute differences (SAD) of corresponding pairs in two arrays. It is a common task in video compression to determine difference between two successive video frames.

Other examples are the Greatest Common Divisor and then Serial-parallel multiplier.

1.2.1 Sum of Absolute Differences

In video compression we might need to quickly determine whether two frames are similar enough to just send difference for second frame.

A technique we can use is **SAD** (*Sum of Absolute Differences*).

As an example, we compare corresponding 16x16 "blocks", treating each 16x16 block as a 256-byte array. From these we compute the absolute value of the difference of each array item, sum those differences:if above a threshold, send complete frame for second frame; if below, can use difference method (using another technique, not described here).

This is most easily described via an algorithm. For instance, in VHDL:

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.std_logic_signed.ALL;

ENTITY SAD IS
  PORT (go: IN std_logic;
  SAD_out: OUT integer;
  Clk, Rst: IN std_logic );
END SAD;

ARCHITECTURE SAD_Alg OF SAD IS
```

```
12   TYPE SADArrayType IS ARRAY(0 TO 255)
13   OF std_logic_vector(7 DOWNTO 0);
14   -- somehow declare A and B here
15 BEGIN
16   PROCESS
17   VARIABLE Sum: integer;
18   BEGIN
19     IF (NOT(go='1')) THEN
20       WAIT UNTIL go='1';
21     END IF;
22
23     Sum := 0;
24     FOR I IN 0 TO 255 LOOP
25       Sum := Sum + Conv_Integer(ABS(A(I) - B(I)));
26     END LOOP;
27
28     WAIT FOR 50 ns; -- model comb. delay
29     SAD_out <= Sum;
30   END PROCESS;
31 END SAD_Alg;
```